

Building your own text-generating AI (1/4)

Lecture 8 – Treating text as data · embeddings · character models

Dr Thomas Robinson

August 12, 2026

Warm-up · three questions from last time

≈ 5 MIN · SHOUT IT OUT

1. What forces an autoencoder to learn a compressed representation rather than copying its input?
2. What can you do with a trained VAE that you cannot do with a plain autoencoder — and which part of the loss makes it possible?
3. In the reparameterisation trick we write $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$. Why not just sample $\mathbf{z}$ directly?

Warm-up · answers

- 1. The bottleneck.** The latent dimension is smaller than the input, so the network cannot pass the input through unchanged – it has to compress.
- 2. Sample new images from the latent space.** The KL term in the VAE loss keeps the latent space organised around the origin, so a point drawn from $\mathcal{N}(\mathbf{0}, \mathbf{I})$ decodes to something sensible. A plain autoencoder has no such guarantee – most latent points decode to nonsense.
- 3. Gradients cannot flow through a random sampling step.** Writing $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$ moves the randomness into $\boldsymbol{\epsilon}$, which has no parameters. The path from the loss back to $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ is now deterministic, so backprop works.

| Part 1 · Text as data

How do we feed text to a neural network?

A neural network takes **numbers**. Text is a sequence of characters / words / morphemes / utf-8 codepoints.

Tokenisation = the choice of *what counts as one unit*. The rest of the model — embeddings, attention, output layer — works on top of that decision.

THREE GRANULARITIES TO CHOOSE FROM

Character

- Tiny vocab (~ 100)
- Long sequences
- Where we're going to start

WORD

- Huge vocab (10^5+)
- Out-of-vocabulary is a hard problem
- What 1990s NLP used

Subword (BPE)

- In-between (~ 50 - $100k$)
- Composable
- What GPT, Claude, BERT use
- (We'll get to this on Day 11)

What does the text actually look like to the model?

The string "hello" is, to your computer, a sequence of **bytes**: [104, 101, 108, 108, 111].

To a language model, after tokenisation, it's a sequence of **token ids** — typically much shorter:

As characters (tokens = chars)

"hello world" → [7, 4, 11, 11, 14, 0, 22, 14, 17, 11, 3] (*ids illustrative*)

11 tokens needed for the string

- Each id picks one row from a small lookup table.

As BPE (tokens = subwords)

"hello world" → [15339, 1917]

Two tokens. Each picks one row from a vocab of ~100,000.

(These are the *actual* ids GPT-4's tokeniser, `cl100k`, assigns — and its vocab is ~100k.)



Note

The choice of tokeniser determines **the cost** (sequence length) and **the unit** the model learns to work with. We'll see both on Day 11.

A baby character tokeniser

Take all unique characters in some corpus. Number them 0, 1, 2,

```
text = "to be or not to be"
chars = sorted(set(text))      # [' ', 'b', 'e', 'n', 'o', 'r', 't']
stoi = {c: i for i, c in enumerate(chars)}
itos = {i: c for c, i in stoi.items()}

encode = lambda s: [stoi[c] for c in s]
decode = lambda ids: "".join(itos[i] for i in ids)
```

- `encode("to be")` → `[6, 4, 0, 1, 2]`.
- Each integer is a **token id**.
- The network will see and produce integers.



And that's all we need!

Once each character/token has a unique integer id, we just need to figure out how to process sequences of them.

Walking through a tokenisation

Corpus: "the cat sat on the mat"

STEP 1 — UNIQUE CHARACTERS

[' ', 'a', 'c', 'e', 'h', 'm', 'n', 'o', 's', 't'] — 10 distinct characters.

STEP 2 — ASSIGN IDS

char		a	c	e	h	m	n	o	s	t
id	0	1	2	3	4	5	6	7	8	9

STEP 3 — ENCODE

"the cat" → [9, 4, 3, 0, 2, 1, 9]

7 ids. The model has *never seen* the letter **t** — it has only ever seen the integer **9**.

We don't want to one-hot encode

You might think to feed token id i as a one-hot vector $\mathbf{e}_i \in \mathbb{R}^V$:

- For $V = 50,000$: every token becomes a 50,000-dim sparse vector (only one entry is a 1)
- **Massive** but also **orthogonal** – the representation encodes *no* similarity at all.

Words have structure. "King" and "queen" should be close. "King" and "saucepan" should not. One-hot encodings know nothing about this.

We want a **dense, learned, low-dimensional** representation. That representation is called an **embedding**.

An embedding is a lookup table

A mapping from token id i to a dense vector $\mathbf{e}_i \in \mathbb{R}^d$.

Implemented as a learnable **embedding matrix** $\mathbf{E} \in \mathbb{R}^{V \times d}$.

- Row i of $\mathbf{E}$ is the embedding of token i .
- Embedding lookup = "select row i ".
- The entries of $\mathbf{E}$ are **just parameters** — learned by backprop, like any other.

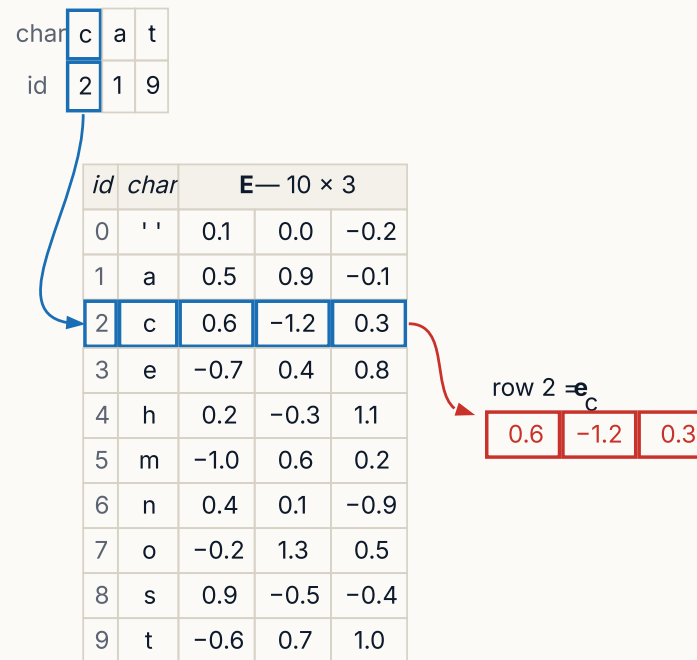
In PyTorch: `nn.Embedding(num_embeddings=V, embedding_dim=d)`.

Example lookup

"the cat sat on the mat"

CORPUS IDS FROM THE TOKENISATION SLIDE

- Embedding dim $d = 3$
- $\mathbf{E}$ is 10×3 .



Step 1 · tokenise: 'c' → id 2 — an integer, nothing more

Step 2 · look up: take row 2 of $\mathbf{E}$ — no arithmetic, just indexing

Step 3 · hand over: $\mathbf{e}_c$ [0.6, -1.2, 0.3] is what the next layer receives

a and **t** then get the same treatment. The 30 numbers in $\mathbf{E}$ are learned parameters.

Why embeddings end up meaningful

The **distributional hypothesis** (Harris, 1954):

Words that occur in similar contexts have similar meanings.

- Train a model to predict words from their context.
- The hidden representation must capture context-relevant similarity.
- Words appearing in interchangeable contexts get nudged to similar embeddings.

Nobody defines “meaning” anywhere in this setup. Similar embeddings fall out of the prediction task on their own.

How embeddings actually get trained

WORD2VEC, IN ONE SLIDE

Skip-gram

Given a centre word, predict its **neighbours** (within a window).

- Input: one word.
- Output: a *single* softmax over the whole vocabulary (V slots), trained to put probability mass on each true neighbour.
- Loss: cross-entropy.

CBOW (continuous bag-of-words)

Given the neighbours, predict the **centre** word.

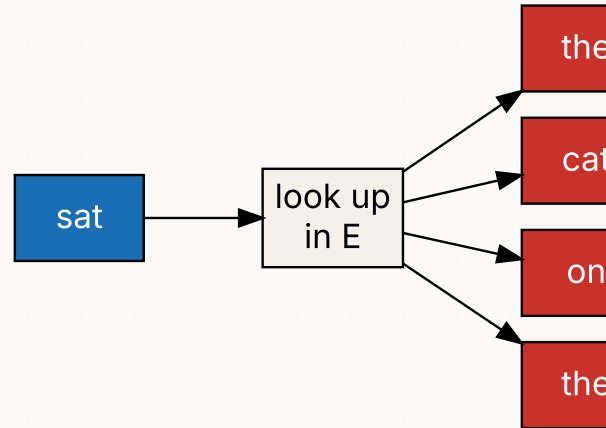
- The inverse problem.
- Slightly more efficient to train.
- Same family of result.

Why this works. To predict context, the model *has* to encode similarity-of-meaning into the embeddings — otherwise it can't generalise across the millions of contexts in the training corpus.

GPT, Claude, and BERT all train embeddings the same way — as a side-effect of their primary task.

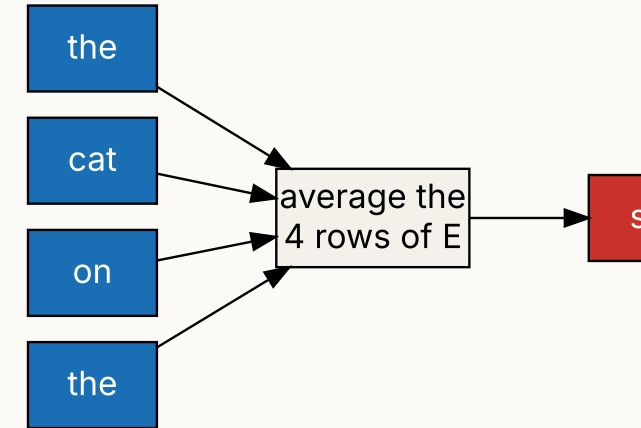
The difference, drawn

Skip-gram



One input, four predictions — this window alone contributes four cross-entropy terms.

CBOW



Four inputs, averaged, one prediction — the averaging is the *bag*: order inside the window is gone.

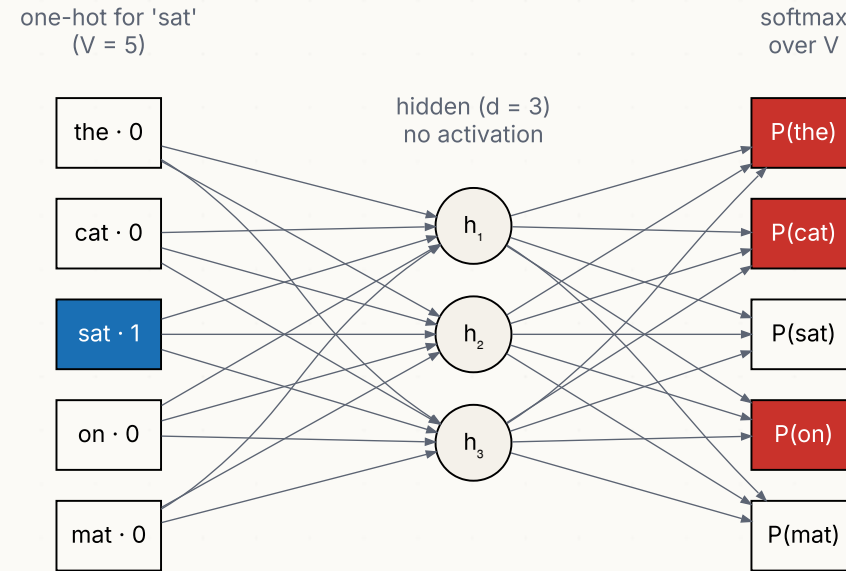


Both arrows point at the same table

Whichever direction you pick, the parameters being trained are the rows of **E**. After millions of windows, **sat**'s row has been nudged towards the rows of the words it keeps company with.

What does the network actually look like?

THE WHOLE OF WORD2VEC (SKIP-GRAM), AT THE SCALE OF OUR FIVE-WORD VOCABULARY. BLUE = INPUT WORD; RED = THE NEIGHBOURS TO PREDICT.



The left bundle of arrows is $\mathbf{W}_{\text{in}}$ (5×3): multiplying the one-hot by it selects one row. So the hidden layer *is* **sat**'s embedding. The right bundle is $\mathbf{W}_{\text{out}}$ (3×5), feeding the softmax.

NUMBER OF PARAMETERS

$\mathbf{W}_{\text{in}}$: $5 \times 3 = 15$. $\mathbf{W}_{\text{out}}$: $3 \times 5 = 15$. Total: **30** (no bias terms!)

At Google News scale ($V = 3\text{M}$, $d = 300$): 900M *per matrix*.

The shapes-table entry earlier was $\mathbf{W}_{\text{in}}$.



At inference, we throw away the output

After training, row i of $\mathbf{W}_{\text{in}}$ is word i 's embedding. We discard $\mathbf{W}_{\text{out}}$ and the softmax.

- The prediction task was used *only* to learn the embeddings

Can we afford that softmax?

THE OUTPUT LAYER, AT REAL SCALE ([MIKOLOV ET AL., 2013](#))

Our toy softmax had 5 slots. Google News word2vec has $V = 3\text{M}$.

THE BILL, PER TRAINING PAIR

To normalise a softmax you must score **every** word: $\mathbf{h} \mathbf{W}_{\text{out}}$ is $300 \times 3\text{M} = 900\text{M}$ multiplications — for *one* window, before *one* update. Multiply by billions of windows.

THE FIX — NEGATIVE SAMPLING

Swap “which of 3M words is the neighbour?” for a handful of **yes/no questions**:

- The real pair from the corpus — (**sat**, **cat**) → target **yes**.
- k fake pairs with randomly drawn words — (**sat**, **quasar**), (**sat**, **invoice**), ... → target **no**.

Each question is a logistic regression on the two words' vectors. With $k = 5$: one update touches $6 \times 300 = 1,800$ output weights, not 900M — **half a million times less work**.

The embeddings come out about as good — and since we discard the output side anyway, nothing of value is lost.

The classic demonstration ([Mikolov et al., 2013](#))

Word2vec embeddings, trained on a big corpus:

$$\mathbf{e}_{\text{king}} - \mathbf{e}_{\text{man}} + \mathbf{e}_{\text{woman}} \approx \mathbf{e}_{\text{queen}}$$

Analogies become vector arithmetic. “King is to man as queen is to woman” — the relationship is encoded as a *direction* in the embedding space.

The model wasn’t trained to do analogies. It learned them as a by-product of predicting context words.



Warning

Real-world embeddings encode *all* statistical regularities of their training corpus — including the unsavoury ones. We come back to this.

Measuring “closeness” with cosine similarity

WHAT DOES THAT $\approx$ ACTUALLY MEAN FOR TWO VECTORS?

We keep saying two embeddings are “close.” The standard measure is **cosine similarity** — the cosine of the angle θ between them:

$$\cos \theta = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$$

- $+1 \rightarrow$ same direction (very similar).
- $0 \rightarrow$ orthogonal (unrelated — exactly the one-hot case).
- $-1 \rightarrow$ opposite direction.

Calculating cosine similarity

A 2-D WORKED EXAMPLE

Let $\mathbf{a} = (3, 4)$ and $\mathbf{b} = (4, 3)$.

- Dot product: $\mathbf{a} \cdot \mathbf{b} = 3 \times 4 + 4 \times 3 = 24$.
- Lengths: $\|\mathbf{a}\| = \sqrt{3^2 + 4^2} = 5$ and $\|\mathbf{b}\| = \sqrt{4^2 + 3^2} = 5$.
- So $\cos \theta = \frac{24}{5 \times 5} = \frac{24}{25} = 0.96$ — nearly the same direction.

Now take $\mathbf{c} = (4, -3)$: $\mathbf{a} \cdot \mathbf{c} = 12 - 12 = 0$, so $\cos \theta = 0$ — **orthogonal**, i.e. unrelated. (This is exactly what one-hot vectors are to each other.)

TRY — 30 SECONDS IN PAIRS

Compute $\cos \theta$ for $\mathbf{a} = (1, 0)$ and $\mathbf{b} = (1, 1)$.

ANSWER

$\mathbf{a} \cdot \mathbf{b} = 1 \times 1 + 0 \times 1 = 1$; $\|\mathbf{a}\| = 1$ and $\|\mathbf{b}\| = \sqrt{2} \approx 1.41$.

$\cos \theta = \frac{1}{1 \times 1.41} \approx 0.71$ — a 45° angle. Halfway between “identical” and “unrelated.”

The analogy, with actual numbers

LET'S EARN THAT $\approx$ IN A TOY 2-D EMBEDDING SPACE

Suppose, in our tiny space, the embeddings are:

$$\mathbf{e}_{\text{king}} = (2, 1), \quad \mathbf{e}_{\text{man}} = (1, 1), \quad \mathbf{e}_{\text{woman}} = (1, 2)$$

STEP 1 — DO THE ARITHMETIC

$$\mathbf{e}_{\text{king}} - \mathbf{e}_{\text{man}} + \mathbf{e}_{\text{woman}} = (2 - 1 + 1, 1 - 1 + 2) = (2, 2)$$

Call this result $\mathbf{r} = (2, 2)$, with length $\|\mathbf{r}\| = \sqrt{2^2 + 2^2} = \sqrt{8} \approx 2.83$.

STEP 2 — FIND THE NEAREST WORD BY COSINE SIMILARITY

We compare $\mathbf{r}$ to three candidate embeddings (each of length 5, so every denominator is $2.83 \times 5 \approx 14.14$):

word	vector $\mathbf{v}$	$\mathbf{r} \cdot \mathbf{v}$	$\cos \theta$
queen	$(4, 3)$	$8 + 6 = 14$	$14/14.14 \approx 0.99$
rock	$(0, 5)$	$0 + 10 = 10$	$10/14.14 \approx 0.71$
fish	$(3, -4)$	$6 - 8 = -2$	$-2/14.14 \approx -0.14$

VERY HIGH SIMILARITY

$\mathbf{e}_{\text{king}} - \mathbf{e}_{\text{man}} + \mathbf{e}_{\text{woman}}$ points almost exactly at **queen** ($\cos \theta \approx 0.99$) — far closer than to *rock* or *fish*.

We can do the same thing with characters

Character-level embeddings work the same way:

- Vocabulary $V = 65$ characters (uppercase + lowercase + punctuation + whitespace).
- Embedding dim $d = 32$, for example
- Embedding matrix: $65 \times 32 = 2,080$ parameters. Next to nothing.

Each character gets a dense vector. Backprop tunes those vectors to be useful for the next-character prediction task.

Embedding shapes you'll see in practice

Model	Vocab V	Dim d	$V \cdot d$
Character LM (today's lab)	65	32	~2k
word2vec (Google News)	3M	300	~900M
BERT-base	30,522	768	~23M
GPT-3	50,257	12,288	~617M
Llama-3-70B	128,256	8,192	~1.05B



What proportion of a model is the embedding matrix?

It depends how big the rest of the model is. GPT-2's ($50,257 \times 768 \approx 38.6\text{M}$) is about a *third* of its 124M parameters whereas Llama-3-70B's is ~1.5%.

Stop and take stock

STOP & CHECK

TRY — 60 SECONDS IN PAIRS

We want a model that predicts next character from current character. We have a vocab of 65 characters and use 32-dim embeddings.

What is the shape of the embedding matrix? And what does each row of it *do* in the model?

ANSWER

Shape: (65, 32).

Each row is **the dense vector representation of one character**. When the model receives character id i , it looks up row i — and uses that vector as the input to subsequent layers.

Rows are learnable. Backprop will pull characters that appear in similar contexts towards similar embeddings.

| Plenary · tokenise & count

Plenary · tokenise + bigram counts

≈ 10 MIN · IN PAIRS

CORPUS

"the cat sat on the mat"

Treat each **character** as a token (including spaces). Treat the end of the string as a special end token.

TASK 1: TOKENISE

1. List all unique characters.
2. Assign each an integer id.
3. Encode the corpus as a list of ids.

TASK 2: BIGRAM COUNTS

A **bigram** is a pair of adjacent characters.

- Go through your encoded corpus.
- For each pair (c_a, c_b) , increment a counter.
- What's the most common bigram? Second most common?

Six minutes.

Plenary · partial answer

UNIQUE CHARACTERS 10 (including the space): ' ', 'a', 'c', 'e', 'h', 'm', 'n', 'o', 's', 't'.

MOST COMMON BIGRAM — ('a', 't'), which appears **three** times: **cat**, **sat**, **mat**. (The pair ('t', 'h') appears only twice — **the**, **the**.)

MOST FREQUENT CHARACTER — 't', counting *every* occurrence, appears **five** times: **the**, **cat**, **sat**, **the**, **mat**.



Tip

This count table *is* a model. If you have $P(c_b \mid c_a)$ for every pair, you can sample text:

- Pick a starting character.
- Sample next character given current.
- Repeat.

It generates gibberish — but **with structure**. Next we'll meet the smallest neural-network version of this.

| **Part 2 · What embeddings mean**

Visualising embeddings

HIGH-DIM VECTORS COMPRESSED TO 2D

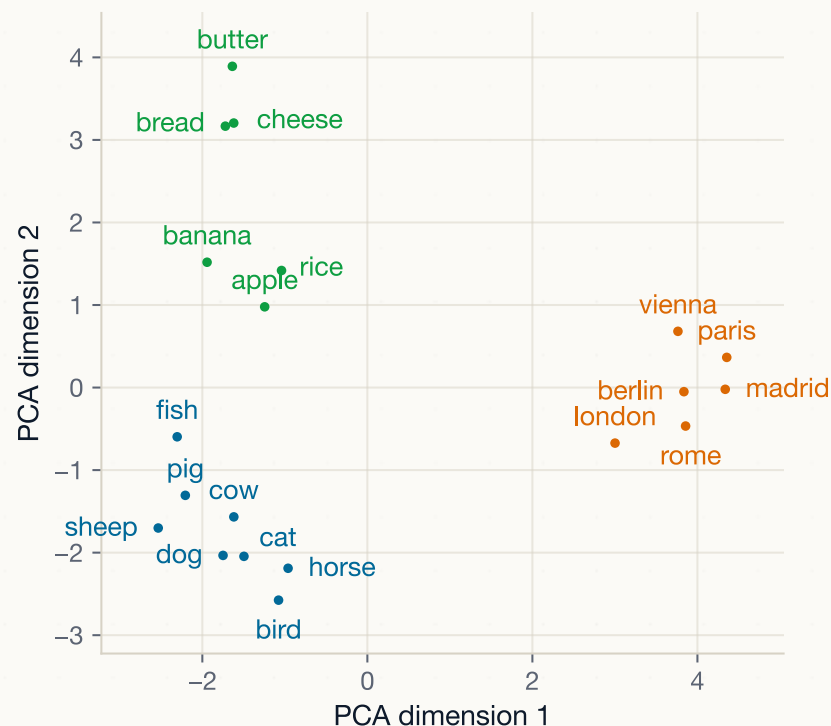
Embeddings live in $\mathbb{R}^{50}$, $\mathbb{R}^{300}$, higher — invisible to the eye.

But we can **project** them down to 2D using:

- **PCA** — fast, linear, preserves global structure.
- **t-SNE** — slow, non-linear, preserves *local* structure (clusters pop out).
- **UMAP** — newer; balances local + global.

On the right: 20 words from **GloVe** (50-d vectors, trained on 6 billion tokens of Wikipedia + newswire), PCA to 2D.

The categories separate on their own — nobody told the model they exist.



The two PCA directions keep 55% of the 50 dimensions' variance. And look at **fish** — pulled off the animal cluster, towards the food one. The corpus knows fish is also dinner (to some).

Static vs contextual embeddings

AN IMPORTANT DISTINCTION

Static

- **One vector per word, period.**
- "bank" in *river bank* and *Bank of England* has the same embedding.
- Word2vec, GloVe, fastText.
- What we use today.

Contextual

- **The vector depends on the sentence.**
- "bank" in two sentences gets *two different* vectors.
- BERT, GPT, Claude all produce these.
- Requires **attention** (Day 10).

Static embeddings are most useful for retrieval, document similarity, social-science measurement. For *generation* you will want to use contextual embeddings.

Embeddings as geometry

Once you have a trained embedding matrix, you can:

- **Cluster** documents by averaging token embeddings within each
- **Search** by embedding query + corpus, returning the nearest neighbour — *semantic retrieval*
 - This is the retrieval step behind **RAG** (retrieval augmented generation)
- **Classify** — feed embeddings into a classifier for sentiment, stance, topic.
- **Compare** documents using cosine similarity.

Embeddings are the *glue* of much of NLP!

From word vectors to document vectors

THE SIMPLEST RECIPE: AVERAGE

To embed a sentence or a whole document: look up every word's vector, take the mean.

A WORKED 2-D EXAMPLE

Suppose $\mathbf{e}_{\text{great}} = (2, 1)$, $\mathbf{e}_{\text{terrible}} = (-2, 1)$, $\mathbf{e}_{\text{film}} = (0, 3)$.

- *"great film"* $\rightarrow \frac{(2, 1) + (0, 3)}{2} = (1, 2)$
- *"terrible film"* $\rightarrow \frac{(-2, 1) + (0, 3)}{2} = (-1, 2)$

The **words** *great* and *terrible* point apart: $\cos \theta = \frac{-4 + 1}{\sqrt{5} \sqrt{5}} = -0.6$.

The **documents** end up close: $\cos \theta = \frac{-1 + 4}{\sqrt{5} \sqrt{5}} = +0.6$ — the shared *film* dominates the average.

- Averaging keeps the **topic** and dilutes the **sentiment** — fine for "what is this about?", risky for "what does it say?"
- Word order is gone too: *"dog bites man"* and *"man bites dog"* average to the same vector.
- The next slide's ideology measure is exactly this recipe. Modern retrieval (RAG) upgrades it with transformer sentence embeddings — Day 10.

Social-science example — measuring concepts

Word embeddings let us measure abstract concepts.

- Train embeddings on a large corpus of legislative speeches
- Compute the average embedding of words like *redistribution, taxation, welfare*
- Compute distance from each MP's speech embedding to that "left-economic" direction
- You get a **scaled measure** of left-economic emphasis per MP

This is how Rodriguez, Spirling & Stewart (2023) and others measure ideology *from text alone*.

A concept that's hard to define directly often has a **geometric expression** in embedding space:

- Direction = the concept itself
- Magnitude = how strongly an item carries the concept
- Cluster = a discovered category

Social-science example — bias in embeddings

Bolukbasi et al. (2016) — *Man is to computer programmer as woman is to homemaker?*

- Word2vec embeddings trained on Google News.
- The vector arithmetic $\mathbf{e}_{\text{man}} - \mathbf{e}_{\text{woman}}$ — call this the “gender direction”.
- For occupations: *doctor* lies closer to “man”, *nurse* to “woman”.
- For adjectives: *brilliant* to “man”, *lovely* to “woman”.

The embeddings are “doing their job”: they reflect how words actually co-occur in the corpus

- Including the patterns we’d prefer they didn’t!

A model trained on these embeddings will inherit the bias.

- E.g. downstream classifiers for résumé screening

What to do about embedding bias

THREE APPROACHES, ALL IMPERFECT

Geometric debiasing

Find the “gender direction” in embedding space and *project it out* of occupation embeddings.

- Fast.
- Removes one axis of bias.
- Bolukbasi et al. (2016).

Data debiasing

Curate or rebalance the training corpus.

- Slow.
- Hard to do completely.
- Hard to even *define* what “balanced” looks like for cultural concepts.

Document and live with it

Be explicit about what the embeddings encode.

- Often the most honest path for research.
- The bias *is the data*; removing it may distort the construct you want to measure.



Bias in social science applications

Using embeddings as **measurement instruments** can cut both ways: it can reveal real corpus patterns (even if biased) *but* it can also distort downstream measurement.

Debiasing is harder than it looks

THE PAPER IS TITLED “LIPSTICK ON A PIG” — THAT’S THE FINDING

Gonen & Goldberg tested the geometric fix from the last slide ([Gonen & Goldberg, 2019](#)):

- Take the words with the strongest gender associations in the original embeddings.
- Apply the standard debiasing projection — find the gender direction, subtract it out.
- Look at the “debiased” space: **those same words still cluster together**, and a classifier can still recover each word’s former gender association from its new embedding.

The bias was never stored in one direction you can subtract away. It is spread across many dimensions of the space.

The practical upshot is that you should **measure and report** the bias in the embeddings you use.

What we'll *do* with embeddings today

We're going to use the smallest possible embedding matrix — one row per character — as the input to a model that predicts the *next character*.

This is called a **bigram language model**

- It is *very bad* at modelling language!
- It helps us motivate the additions we'll need to grow this into GPT

| **Activity · embedding behaviour**

Activity · predict embedding behaviour

≈ 8 MIN · IN PAIRS

A word2vec model is trained on a 10-billion-token English corpus. We extract embeddings of these words:

- king, queen, prince, princess
- cat, dog, horse, fish, table, chair
- London, Paris, Berlin, Rome
- walking, running, swimming, eating

WITHOUT COMPUTING ANYTHING, PREDICT THE ANSWERS

1. Which pair is *closest* in embedding space — (king, queen) or (king, fish) ?
2. What vector arithmetic captures “king is to queen as ? is to princess”?
3. What set of words would you expect to be tightly clustered, separate from the others?
4. Where would $\text{walking} + \text{cat} - \text{dog}$ land — closer to which other word?

Discuss in pairs. We'll compare guesses.

Activity · plausible answers

1. **(king, queen)** — same domain, same level of formality. (king, fish) cross-domain.
2. **king - queen = ? - princess** → ? = prince.
3. **Cities** typically cluster tightly. Animals also, separately. Verbs of locomotion (walking, running, swimming) form another cluster — and the model often learns this without being told.
4. **walking + (cat - dog)** lands close to **walking** itself. *Cat* and *dog* are near neighbours, so their difference is a short vector — adding it barely moves you.



Tip

Different models give slightly different geometry, so the details vary

| Part 3 · The bigram language model

The task

GIVEN THE PREVIOUS CHARACTER, PREDICT THE NEXT CHARACTER.

Train on a chunk of text

- Shakespeare, news, code, etc.

And learn the distribution $P(c_{t+1} \mid c_t)$.

This is the same task GPT solves, just on the smallest possible context (one character).

We *don't need labels* as *the text* acts as its own signal

- This is what makes language modelling so scalable

A pure-count bigram model

Forget neural networks for a moment. The simplest model:

1. Count, for each pair (c_a, c_b) , how often c_b follows c_a .
2. Normalise to get $P(c_b \mid c_a) = \text{count}(c_a, c_b) / \text{count}(c_a)$.
3. To generate: pick a starting character. Sample next from $P(\cdot \mid c_a)$. Repeat.



Note

This is a fine baseline but the output will be awful: e.g. "hewenghadowt h".

- Just notice it has the right *shape* and uses the right *signal*

The whole model, on one slide

THE COUNT TABLE FOR THE PLENARY CORPUS — EVERY CELL OF IT

		next character									
		l	a	c	e	h	m	n	o	s	t
current character	l	0	0	1	0	0	1	0	1	1	1
	a	0	0	0	0	0	0	0	0	0	3
	c	0	1	0	0	0	0	0	0	0	0
	e	2	0	0	0	0	0	0	0	0	0
	h	0	0	0	2	0	0	0	0	0	0
	m	0	1	0	0	0	0	0	0	0	0
	n	1	0	0	0	0	0	0	0	0	0
	o	0	0	0	0	0	0	1	0	0	0
	s	0	1	0	0	0	0	0	0	0	0
	t	2	0	0	0	2	0	0	0	0	0

- Rows: current character. Columns: next. The corpus has **21 adjacent pairs**; every one is tallied in exactly one cell.
- Read row **t**: after **t** we saw **h** twice and **l** twice, so $P(h | t) = \frac{2}{4} = 0.5$.
- Row **a** is fully committed: all three counts in one cell, so $P(t | a) = 1$.

85 of the 100 cells are zero. Hold that thought for the next slide.

The zero-count problem

Take our starting sentence "the cat sat on the mat".

In this corpus, **h** appears twice — and both times the next character is **e**.

So the count table says:

- $\text{count}(\mathbf{h}, \mathbf{e}) = 2$, and $P(\mathbf{e} \mid \mathbf{h}) = 2/2 = 1$.
- $\text{count}(\mathbf{h}, \mathbf{a}) = 0$, and $P(\mathbf{a} \mid \mathbf{h}) = 0/2 = 0$.

TRY IT YOURSELF

What probability does the model give the string "the cat sat on the hat" ?

ANSWER

Zero. Since **h** → **a** never occurs in training, it has probability 0.

- The model multiplies the transition probabilities together, and one zero blanks the whole product

The model confuses *"I never saw this"* with *"this is impossible"*.

Add-one (Laplace) smoothing

We can pretend we see every pair **one extra time**.

$$P(c_b \mid c_a) = \frac{\text{count}(c_a, c_b)}{\text{count}(c_a)} \quad \longrightarrow \quad P(c_b \mid c_a) = \frac{\text{count}(c_a, c_b) + 1}{\text{count}(c_a) + V}$$

The $+V$ in the denominator (here $V = 10$ characters) keeps each row summing to 1.

WORKED EXAMPLE — THE **h** ROW

$\text{count}(\mathbf{h}) = 2$, so every denominator is $2 + 10 = 12$:

- $P(\mathbf{e} \mid \mathbf{h}) = \frac{2 + 1}{2 + 10} = \frac{3}{12} = 0.25$
- $P(\mathbf{a} \mid \mathbf{h}) = \frac{0 + 1}{2 + 10} = \frac{1}{12} \approx 0.083$ — and likewise for the other 8 unseen characters.

Row sum: $\frac{3}{12} + 9 \times \frac{1}{12} = \frac{12}{12} = 1$. Still a probability distribution!

The zero became small-but-nonzero. **"the cat sat on the hat"** now gets a low probability instead of no probability.

Why not trigrams?

If one character of context is good, surely two is better — and ten better still?

THE TABLE BLOWS UP

With $V = 27$ characters (26 letters + space), a count table over n -grams has V^n cells:

context	table size
bigram	$27^2 = 729$
trigram	$27^3 = 19,683$
10-gram	$27^{10} \approx 2 \times 10^{14}$

The problem compounds if we use words: with $V = 50,000$ words, even the **bigram** table has $50,000^2 = 2.5 \times 10^9$ cells – nearly all of them have count 0 (i.e. are sparse).

Counting cannot scale to long contexts whereas sharing parameters through embeddings and a network can.

The same bigram as a neural network

Exactly the same model:

```
import torch.nn as nn

class Bigram(nn.Module):
    def __init__(self, vocab_size):
        super().__init__()
        # one row per character; one column per next-character logit
        self.token_table = nn.Embedding(vocab_size, vocab_size)

    def forward(self, idx):
        # idx is a (B, T) tensor of token ids
        logits = self.token_table(idx)      # (B, T, V)
        return logits
```

- The embedding **is** the logit table. Row i gives the score of each next character given character i .
- Train with cross-entropy on the next character.
- Total parameters: $V \cdot V$ — for $V = 65$, that's 4,225 numbers. Trivial.

This is the smallest substantive language model.

Training the bigram, concretely

A WORKED STEP

- Take one training pair: input 't' (id 9), target 'h' (id 4).
- Look up row 9 of the embedding/logits table → a vector of V scores.
- Apply softmax → a probability over {'', a, c, e, h, m, n, o, s, t}.
- Loss = $-\log P('h') = -\log(\text{slot } 4)$.
- Backprop adjusts row 9 to *raise* the score at slot 4.



Tip

That's it! The model learns “when you see t, weight h a little higher.” If we repeat this over the whole corpus, the bigram statistics emerge in the weights

- These just are pure-count numbers (with a small amount of noise)

Generating from the model

```
def generate(model, start, max_new=200):  
    idx = encode(start)  
    for _ in range(max_new):  
        logits = model(idx[-1:])          # only need the last char  
        probs = softmax(logits, dim=-1)  
        next_id = sample(probs)  
        idx.append(next_id)  
    return decode(idx)
```

- Pass current context → logits → softmax → sample.
- Append the sampled id to the running sequence.
- Repeat.

Sampling is what gives generation its variety.

- We can now vary the *temperature* to set how “adventurous” or “creative” the samples are

Temperature in sampling

A KNOB ON HOW "CREATIVE" THE MODEL IS

Standard softmax: $p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$.

With temperature τ : $p_i = \frac{e^{z_i/\tau}}{\sum_j e^{z_j/\tau}}$.

Low $\tau (< 1)$

- Sharpens the distribution.
- Best token gets even more probability mass.
- Output: confident, repetitive.
- At $\tau \rightarrow 0$: argmax (always pick best).

HIGH $\tau (> 1)$

- Flattens the distribution.
- Rare tokens get more chance.
- Output: creative, sometimes nonsense.

Chat products often (used to) expose temperature as a hyperparameter: change the temperature and the output reads differently, without needing to change the weights.

How good is a language model?

How do we *score* a model?

Walk through a held-out string. At each position, ask the model: *what probability did you give the character that actually came next?*

- Take $-\log p$ of each of those probabilities — the **negative log-likelihood** (NLL). High probability $\rightarrow$ small NLL.
- Average the NLLs over the N positions in the string.
- **Perplexity** is e to that average:

$$\text{perplexity} = e^{\frac{1}{N} \sum_{t=1}^N -\log p_t}$$

Perplexity reads as: *"the model is as unsure as if it were choosing uniformly among k options."*

A model that spreads probability uniformly over 27 characters gives every true character $p = \frac{1}{27}$, so the mean NLL is $-\log \frac{1}{27} = \log 27 \approx 3.30$, and perplexity is $e^{3.30} = 27$. As unsure as a 27-way guess.

Perplexity — a worked example

A model reads a 4-token string. The probabilities it gave the four *true* next tokens:

0.5, 0.25, 0.5, 0.25

STEP 1 — TURN EACH PROBABILITY INTO A NUMBER OF OPTIONS

A fair coin gets the answer right with $p = 0.5$ — so $p = 0.5$ means *as unsure as a 2-way guess*. In general: $1/p$ options.

$0.5 \rightarrow 2$ ways, $0.25 \rightarrow 4$ ways, $0.5 \rightarrow 2$, $0.25 \rightarrow 4$

STEP 2 — AVERAGE THE GUESS SIZES

Multiply them and take the 4th root — the *geometric* mean, the right average for numbers that multiply, as probabilities do:

$$\text{perplexity} = (2 \times 4 \times 2 \times 4)^{1/4} = 64^{1/4} \approx 2.83$$

2.83 options. The model made 2-way guesses on half the tokens and 4-way guesses on the other half, so its average uncertainty lands in between.

The formula on the last slide is this same computation, done with logs: mean NLL = $\frac{0.693+1.386+0.693+1.386}{4} \approx 1.04$, and $e^{1.04} \approx 2.83$. Logs just turn “multiply, then take the root” into “average, then exponentiate”.

Perplexity — get a feel for the scale

TRY IT YOURSELF

What is the perplexity of a model that always puts probability 1 on the correct token? And of random guessing over 27 characters?

ANSWER

- Always right: every NLL is $-\log 1 = 0$, so the mean is 0 and perplexity is $e^0 = \mathbf{1}$. I.e. no uncertainty at all.
- Random guessing: every true token gets $p = \frac{1}{27}$, so perplexity is **27**



Tip

Compute your lab bigram's perplexity this afternoon and we can see whether tomorrow's RNN beats it.

What the bigram model is missing

What it has

- A tokeniser.
- An embedding.
- A loss.
- A training loop.
- A sampler.

All the pieces of a real language model.

What it lacks

- **Context** — only sees the previous character.
- No structure connecting positions.
- No way to handle long-range dependencies (rhyme, names, narrative).

This is why we need recurrence (tomorrow) and attention (Monday).

Stop and take stock

WHERE YOU ARE AT THE END OF DAY 8

You can:

1. Build a character-level tokeniser in 5 lines of Python.
2. Explain why one-hot encoding doesn't scale.
3. Describe an embedding as a learnable lookup table.
4. State the distributional hypothesis and give one CS and one social-science example of its use.
5. Train a bigram language model — and say what it is missing.
6. Compute a perplexity and say what the number means.

A check-in on the headline promise

TWO-THIRDS OF THE WAY THROUGH — WHERE WE ARE

-  Forward pass + backprop
-  Feed-forward classifier
-  Vision (CNNs) + generative (VAE)
-  Text as data + a working bigram language model
-  Tomorrow: memory (RNNs).
-  Monday: **attention** — the last big architectural idea of the course
-  Tuesday: BPE + sampling — your tiny GPT will write alright English

Eight days down, four to go!

| **Lab brief · today's afternoon**

Today's lab — 90 minutes

GOALS

Train a character-level **bigram** language model on a small text corpus.

Steps:

1. Download a small text file (we'll provide tinyshakespeare or similar).
2. Build a character-level tokeniser (encode + decode).
3. Define a **Bigram** model in PyTorch (5–10 lines).
4. Train for 1–2k steps with Adam + cross-entropy.
5. Generate a 200-character sample. Read it. Laugh.

STRETCH GOALS

- Sample at different **temperatures** — what changes?
- Look at the trained embedding matrix. Does it have any visible structure (rows for vowels, etc.)?
- Train on *your own* corpus — your essays, a journal — generate text in that voice.



Tip

The output should be **terrible**. That's the point.

We give the model a memory

Today's bigram sees only the **previous character** — it can't remember how a word/sentence began. Tomorrow we'll give the network a *memory* of everything it has read so far.

Optional tonight: Karpathy, *The Unreasonable Effectiveness of Recurrent Neural Networks* (~20 min). Keep your bigram sample; we'll race it against an RNN tomorrow.

Gonen, H., & Goldberg, Y. (2019). Lipstick on a pig: Debiasing methods cover up systematic gender biases in word embeddings but do not remove them. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 609–614.

Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., & Dean, J. (2013). Distributed representations of words and phrases and their compositionality. *Advances in Neural Information Processing Systems*, 26.